DeepEval

The Open-Source LLM Evaluation Framework

Complete Step-by-Step Study Notes

GitHub	Python	Install	Focus
confident-ai/deepeval	≥ 3.9 required	pip install deepeval	LLM-as-a-Judge

1. What Is DeepEval?

DeepEval is an open-source evaluation framework purpose-built for testing Large Language Model (LLM) applications. Think of it as Pytest — but specifically designed for AI systems. Rather than checking for specific string outputs, DeepEval uses LLM-as-a-judge techniques and statistical NLP models to assess qualities like correctness, relevance, and safety.

> **Core Philosophy**
> DeepEval treats LLM evaluation as software testing. Every AI output can be measured, tracked, and improved — just like any software component.

1.1 Why Evaluate LLMs?

LLMs are probabilistic — the same prompt can return different outputs, and models can hallucinate, produce biased content, or fail silently. Without rigorous evaluation:

- You cannot detect prompt regressions when you change a system prompt
- You cannot compare model versions objectively
- You cannot ensure RAG pipelines retrieve and use context faithfully
- You cannot ship AI features to production confidently

1.2 Key Differentiators

Feature	What It Means
Pytest-style API	Write LLM tests just like unit tests — familiar, composable, CI-ready
LLM-as-a-Judge	Uses an evaluator LLM to score outputs on custom criteria
Local NLP Models	Some metrics run entirely on-device — no API cost for every eval

40+ Metrics	Ready-made metrics for RAG, agents, chat, hallucination, and more
Synthetic Dataset Gen	Auto-generate test cases from your documents
CI/CD Integration	Works seamlessly with GitHub Actions, Jenkins, etc.
Confident AI Platform	Optional cloud dashboard for reports, traces, and team sharing

2. Installation & Setup

2.1 Prerequisites

Requirement	Details
Python Version	3.9 or higher
LLM API Key	OpenAI key recommended for evaluator model (can use others)
Package Manager	pip (standard Python package manager)

2.2 Install DeepEval

Run the following command in your terminal to install or upgrade DeepEval:

```
pip install -U deepeval
```

> **Ő Tip**
> The -U flag ensures you always install the latest version. DeepEval is actively maintained and releases updates frequently.

2.3 Set Your API Key

DeepEval uses an LLM to power its evaluation metrics. By default it uses OpenAI's GPT models. Set your key as an environment variable:

```
# On Mac/Linux
export OPENAI_API_KEY="sk-..."
```

```
# On Windows (Command Prompt)
set OPENAI_API_KEY=sk-...
```

> **? Security Note**
> Never hardcode API keys in source files. Use environment variables or a .env file with python-dotenv. Add .env to your .gitignore.

2.4 (Optional) Create a Confident AI Account

DeepEval has an optional cloud companion called Confident AI. It stores test runs, generates shareable reports, and lets you visualize evaluation trends over time.

```
deepeval login
```

Follow the CLI prompts to create a free account, copy your API key, and paste it in. Once connected, every deepeval test run is automatically uploaded and visible in your dashboard.

3. Core Concepts

3.1 LLMTestCase

The LLMTestCase is DeepEval's fundamental unit. It represents a single interaction with your LLM application, containing:

Field	Description
input	The user's question or prompt sent to the LLM
actual_output	What your LLM application actually returned
expected_output	The ideal/reference answer (optional, used by some metrics)
context	Background facts the model should know (for hallucination checks)
retrieval_context	Chunks retrieved by a RAG pipeline (for RAG metrics)

Example usage:

```
from deepeval.test_case import LLMTestCase
```

```
test_case = LLMTestCase(
    input="What is DeepEval?",
    actual_output="DeepEval is an LLM evaluation framework.",
    expected_output="An open-source framework for evaluating LLM apps.",
    retrieval_context=["DeepEval helps developers test LLM systems."]
)
```

3.2 Metrics

Metrics are the scoring functions that evaluate your test case. Each metric:

- Receives an LLMTestCase as input
- Produces a score between 0.0 and 1.0
- Includes a textual reason/explanation for the score
- Passes or fails based on a configurable threshold

Every metric has this general structure:

```
from deepeval.metrics import AnswerRelevancyMetric
```

```
metric = AnswerRelevancyMetric(
    threshold=0.7,     # Score must be >= 0.7 to pass
    model="gpt-4o"    # Evaluator LLM (optional override)
)
```

3.3 assert_test and evaluate

assert_test integrates with Pytest and raises an error if any metric fails. **evaluate** runs metrics without raising, returning a results object ideal for scripts and notebooks.

```
from deepeval import assert_test, evaluate
```

```
# In pytest — raises AssertionError on failure
assert_test(test_case, [metric1, metric2])
```

```
# In scripts/notebooks — returns results
results = evaluate([test_case], [metric1, metric2])
```

3.4 EvaluationDataset

An EvaluationDataset is a collection of LLMTestCases. You can load it from CSV, HuggingFace, or Confident AI, then run all cases through your metrics in one call.

```
from deepeval.dataset import EvaluationDataset
```

```
dataset = EvaluationDataset(test_cases=[test1, test2, test3])
dataset.evaluate([metric1, metric2])
```

4. Writing Your First Test (Step by Step)

> **ϕ Scenario**
> You have a customer support chatbot that answers questions about your return policy. You want to verify its answers are correct and relevant.

Step 1 — Create the test file

```
touch test_chatbot.py
```

Step 2 — Import dependencies

```
import pytest
from deepeval import assert_test
from deepeval.metrics import GEval, AnswerRelevancyMetric
from deepeval.test_case import LLMTestCase, SingleTurnParams
```

Step 3 — Define your metrics

```
correctness_metric = GEval(
    name="Correctness",
    criteria="Is the actual output factually correct based on the expected output?",
    evaluation_params=[
        SingleTurnParams.ACTUAL_OUTPUT,
        SingleTurnParams.EXPECTED_OUTPUT
    ],
    threshold=0.5
)
```

```
relevancy_metric = AnswerRelevancyMetric(threshold=0.7)
```

Step 4 — Write the test function

```
def test_return_policy():
    test_case = LLMTestCase(
        input="What if these shoes don't fit?",
        actual_output="You have 30 days to get a full refund at no extra cost.",
        expected_output="We offer a 30-day full refund at no extra costs.",
        retrieval_context=[
            "All customers are eligible for a 30-day full refund at no extra costs."
        ]
    )
    assert_test(test_case, [correctness_metric, relevancy_metric])
```

Step 5 — Run the test

```
deepeval test run test_chatbot.py
```

Expected Output

DeepEval will print a detailed table showing each metric's score, pass/fail status, and the LLM's reasoning for the score. A green checkmark means the test passed.

5. Metrics Deep Dive

5.1 Custom / All-Purpose Metrics

Metric	Category	What It Evaluates
G-Eval	*Custom*	Evaluates any custom criteria using LLM-as-judge. Research-backed. Most flexible metric.
DAG	*Custom*	Graph-based deterministic evaluation using a decision tree of LLM checks.

5.2 RAG (Retrieval-Augmented Generation) Metrics

Metric	Category	What It Evaluates
Answer Relevancy	*RAG*	Is the model's answer relevant to the user's question?
Faithfulness	*RAG*	Does the answer align with the retrieved context? Detects hallucination.
Contextual Recall	*RAG*	Does the retrieval context cover the expected answer?
Contextual Precision	*RAG*	Are the most relevant context chunks ranked at the top?
Contextual Relevancy	*RAG*	How relevant is the overall retrieval context to the input?
RAGAS	*RAG*	Composite average of the 4 core RAG metrics above.

5.3 Agentic Metrics

Metric	Category	What It Evaluates
Task Completion	*Agent*	Did the agent successfully accomplish the given task?
Tool Correctness	*Agent*	Were the right tools called with the right arguments?
Step Efficiency	*Agent*	Did the agent avoid unnecessary or redundant steps?
Plan Adherence	*Agent*	Did the agent follow the expected execution plan?
Goal Accuracy	*Agent*	How accurately was the intended goal achieved?

5.4 Other Key Metrics

Metric	Category	What It Evaluates
Hallucination	*Safety*	Does the output contradict provided factual context?
Bias	*Safety*	Does the output exhibit gender, racial, or political bias?
Toxicity	*Safety*	Does the output contain harmful or offensive language?
Summarization	*Quality*	Is the summary accurate and sufficiently complete?
JSON Correctness	*Format*	Does the output match the expected JSON schema?
Prompt Alignment	*Format*	Does the output follow all instructions in the prompt?

6. G-Eval — The Most Powerful Metric

G-Eval is DeepEval's flagship metric, based on the research paper "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment." It allows you to define any evaluation criteria in natural language and have an LLM judge score your outputs.

How G-Eval Works

G-Eval sends the evaluation criteria, test case fields, and the LLM's actual output to a judge LLM. The judge returns a score (0–1) and a detailed reasoning. This mimics how a human expert would grade the output.

6.1 SingleTurnParams — What You Can Evaluate On

Parameter	What It Represents
ACTUAL_OUTPUT	The LLM's response (always include this)
EXPECTED_OUTPUT	The reference/ideal answer
INPUT	The user's original question
CONTEXT	Background facts the model was given
RETRIEVAL_CONTEXT	The chunks retrieved by a RAG pipeline

6.2 G-Eval Examples

Example A: Custom Correctness

```
correctness = GEval(
    name="Correctness",
    criteria="Determine whether the actual output is factually
              correct based on the expected output.",
    evaluation_params=[
        SingleTurnParams.ACTUAL_OUTPUT,
        SingleTurnParams.EXPECTED_OUTPUT
    ],
    threshold=0.5
)
```

Example B: Tone Evaluation

```
tone = GEval(
    name="Professional Tone",
    criteria="Evaluate whether the response uses a professional
              and helpful tone appropriate for customer support.",
    evaluation_params=[SingleTurnParams.ACTUAL_OUTPUT],
    threshold=0.7
)
```

Example C: Conciseness

```
conciseness = GEval(
    name="Conciseness",
    criteria="Is the response concise and free of unnecessary
              verbosity while still being complete?",
    evaluation_params=[
        SingleTurnParams.INPUT,
        SingleTurnParams.ACTUAL_OUTPUT
    ],
    threshold=0.6
)
```

7. Evaluating RAG Pipelines

RAG (Retrieval-Augmented Generation) systems have two failure modes: bad retrieval (finding the wrong context) and bad generation (not using the context faithfully). DeepEval has dedicated metrics for both.

> **‘ RAG Failure Modes**
> Retrieval failure: The wrong documents are fetched → low Contextual Relevancy / Recall. Generation failure: The right docs are fetched but the model ignores or distorts them → low Faithfulness.

7.1 Full RAG Test Example

```
from deepeval.metrics import (
    AnswerRelevancyMetric,
    FaithfulnessMetric,
    ContextualRecallMetric,
    ContextualPrecisionMetric,
    ContextualRelevancyMetric
)
```

```
test_case = LLMTestCase(
    input="What are the symptoms of diabetes?",
    actual_output="Common symptoms include excessive thirst, frequent urination,
                   and fatigue.",
    expected_output="Diabetes symptoms: frequent urination, excess thirst,
                     unexplained weight loss, fatigue, blurred vision.",
    retrieval_context=[
        "Type 2 diabetes symptoms: polyuria, polydipsia, fatigue.",
        "Blurred vision and slow-healing sores can also indicate diabetes."
    ]
)
```

```
metrics = [
    AnswerRelevancyMetric(threshold=0.7),
    FaithfulnessMetric(threshold=0.7),
    ContextualRecallMetric(threshold=0.6),
    ContextualPrecisionMetric(threshold=0.6),
]
```

```
assert_test(test_case, metrics)
```

8. Synthetic Test Data Generation

One of DeepEval's most powerful features is its ability to automatically generate test cases from your own documents. This is ideal when you have a knowledge base (PDFs, text files) but no labeled QA pairs.

8.1 How It Works

1. You provide source documents (PDFs, text files, raw strings)
2. DeepEval's Synthesizer uses an LLM to generate input questions
3. It also generates expected outputs and retrieval contexts
4. You get a ready-to-use EvaluationDataset

8.2 Code Example

```
from deepeval.synthesizer import Synthesizer
from deepeval.dataset import EvaluationDataset
```

```
synthesizer = Synthesizer()
```

```
# Generate from raw text documents
goldens = synthesizer.generate_goldens_from_docs(
    document_paths=["./knowledge_base.pdf", "./faq.txt"],
    max_goldens_per_document=10
)
```

```
# Create a dataset from the generated goldens
dataset = EvaluationDataset(goldens=goldens)
dataset.save_as_csv("test_dataset.csv")
```

> **Ő Golden vs Test Case**
> A Golden is like a test template — it contains an input and expected output but no actual output yet. When you run your LLM on a Golden's input, the result becomes a full LLMTestCase.

9. CI/CD Integration

DeepEval integrates natively with any CI/CD pipeline. Since it uses the Pytest runner, you can add LLM evaluation as a quality gate to your pull requests and deployments.

9.1 GitHub Actions Example

```
# .github/workflows/llm-eval.yml
name: LLM Evaluation

on: [push, pull_request]

jobs:
  evaluate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - run: pip install deepeval
      - run: deepeval test run tests/test_chatbot.py
        env:
          OPENAI_API_KEY: ${{ secrets.OPENAI_API_KEY }}
          DEEPEVAL_API_KEY: ${{ secrets.DEEPEVAL_API_KEY }}
```

Й Quality Gate

With this setup, any PR that causes an LLM metric to drop below threshold will fail the CI check — preventing regressions from reaching production.

10. Building Custom Metrics

When the built-in metrics don't cover your specific use case, DeepEval lets you build your own. Custom metrics inherit from BaseMetric and integrate fully with the DeepEval ecosystem (CI/CD, Confident AI dashboard, etc.).

10.1 Custom Metric Structure

```
from deepeval.metrics import BaseMetric
from deepeval.test_case import LLMTestCase
```

```
class MyCustomMetric(BaseMetric):
    def __init__(self, threshold: float = 0.5):
        self.threshold = threshold
        self.name = "My Custom Metric"
```

```
    def measure(self, test_case: LLMTestCase) -> float:
        # Your custom scoring logic here
        output = test_case.actual_output
        score = len(output) / 500  # simple example
        self.score = min(score, 1.0)
        self.reason = f"Output length score: {self.score:.2f}"
        self.success = self.score >= self.threshold
        return self.score
```

```
    async def a_measure(self, test_case: LLMTestCase) -> float:
        # Async version — required for concurrent evaluation
        return self.measure(test_case)
```

```
    def is_successful(self) -> bool:
        return self.success
```

11. Framework Integrations

DeepEval integrates with all major LLM frameworks via lightweight wrappers and callback handlers, enabling evaluation without changing your application logic.

Framework	How to Integrate
OpenAI	Wrap the client: from deepeval.openai import OpenAI
Anthropic	Wrap the client: from deepeval.anthropic import Anthropic
LangChain	Use CallbackHandler from deepeval.integrations.langchain
LangGraph	Same CallbackHandler works for LangGraph agents
OpenAI Agents	Auto-traced — wrap once, evaluate entire agent runs
LlamaIndex	Callback integration for RAG pipeline evaluation
CrewAI	Evaluate multi-agent system outputs end-to-end
Pydantic AI	Type-safe agent evaluation with validation

11.1 Anthropic Integration Example

```
from deepeval.anthropic import Anthropic
from deepeval.tracing import trace
from deepeval.metrics import TaskCompletionMetric
```

```
client = Anthropic()
```

```
for golden in dataset.evals_iterator():
    with trace(metrics=[TaskCompletionMetric()]):
        client.messages.create(
            model="claude-sonnet-4-5",
            max_tokens=1024,
            messages=[{"role": "user", "content": golden.input}],
        )
```

12. LLM Benchmarking

DeepEval includes support for industry-standard LLM benchmarks. You can evaluate any model on these benchmarks in under 10 lines of code — useful for model selection and comparison.

Benchmark	What It Tests
MMLU	Multitask Language Understanding — 57 academic subjects
HellaSwag	Commonsense NLI — completing plausible sentence continuations
HumanEval	Code generation — solving Python programming problems
GSM8K	Grade school math — multi-step arithmetic reasoning
TruthfulQA	Truthfulness — resisting common misconceptions
DROP	Discrete Reasoning Over Paragraphs — arithmetic over text
BIG-Bench Hard	Challenging tasks where models typically underperform humans

12.1 Quick Benchmark Example

```
from deepeval.benchmarks import MMLU
from deepeval.benchmarks.tasks import MMLUTask
```

```
benchmark = MMLU(
    tasks=[MMLUTask.HIGH_SCHOOL_MATHEMATICS, MMLUTask.ASTRONOMY],
    n_shots=5
)
```

```
benchmark.evaluate(model="gpt-4o")
print(benchmark.overall_score)
```

13. Best Practices & Pro Tips

13.1 Choosing the Right Metric

Use Case	Recommended Metrics
RAG Chatbot	Answer Relevancy + Faithfulness + Contextual Recall
AI Agent	Task Completion + Tool Correctness + Step Efficiency
Summarization	Summarization + Faithfulness
Code Generation	JSON Correctness + G-Eval (custom correctness)
Customer Support Bot	G-Eval (tone) + Answer Relevancy + Bias
Multi-turn Chat	Knowledge Retention + Role Adherence + Turn Relevancy

13.2 Threshold Tuning

- Start with threshold=0.5 while developing — tighten as your system improves
- Use 0.7–0.8 for production quality gates
- Faithfulness should be high (≥ 0.8) for healthcare/legal applications
- Run 20–50 test cases before finalizing thresholds to understand score distribution

13.3 Evaluation Cost Management

- Use local NLP models for simple checks (toxicity, JSON correctness)
- Reserve GPT-4o for critical quality metrics like G-Eval and Faithfulness
- Batch evaluations — use evaluate() instead of multiple assert_test() calls
- Cache evaluation results using Confident AI to avoid re-running identical cases

13.4 Test Case Design

- Include edge cases: ambiguous inputs, very short/long answers, foreign languages
- Cover both positive cases (correct answers) and negative cases (wrong answers)
- For RAG: include test cases where the answer is NOT in the context (faithfulness test)
- Version your test datasets alongside your code in git

Common Pitfall

Don't evaluate with the same LLM that powers your application. Use a different, preferably stronger model as the judge (e.g., evaluate GPT-3.5 apps with GPT-4o) to avoid self-serving bias.

14. Quick Reference Cheat Sheet

14.1 Essential Commands

Command	Purpose
pip install -U deepeval	Install or upgrade DeepEval
deepeval login	Connect to Confident AI cloud platform
deepeval test run test_file.py	Run evaluation tests
deepeval test run test_file.py -n 4	Run tests in parallel (4 workers)
deepeval test run --ignore-errors	Continue despite errors in individual cases
deepeval set-azure-openai ...	Configure Azure OpenAI as the evaluator

14.2 Import Cheat Sheet

```
# Core
from deepeval import assert_test, evaluate
from deepeval.test_case import LLMTestCase, SingleTurnParams
from deepeval.dataset import EvaluationDataset
```

```
# Metrics
from deepeval.metrics import (
    GEval, AnswerRelevancyMetric, FaithfulnessMetric,
    ContextualRecallMetric, ContextualPrecisionMetric,
    HallucinationMetric, BiasMetric, ToxicityMetric,
    SummarizationMetric, TaskCompletionMetric
)
```

```
# Data Generation
from deepeval.synthesizer import Synthesizer
```

```
# Tracing
from deepeval.tracing import observe, update_current_span
```

14.3 Score Interpretation

Score Range	Interpretation
0.0 – 0.3	Poor — significant issues, likely failing tests
0.3 – 0.5	Mediocre — noticeable problems, borderline quality
0.5 – 0.7	Acceptable — reasonable quality for development
0.7 – 0.9	Good — production-worthy for most use cases

0.9 – 1.0	Excellent — near-perfect, high-confidence quality

15. Summary

DeepEval provides a comprehensive, developer-friendly framework for evaluating LLM applications. Here are the key takeaways:

5. DeepEval works like Pytest — write test functions, run with deepeval test run
6. LLMTestCase is the core unit: input, actual_output, expected_output, retrieval_context
7. 40+ metrics cover RAG, agents, chat, safety, and custom criteria
8. G-Eval is the most flexible — define any criteria in plain English
9. RAG pipelines need at least: Faithfulness + Answer Relevancy + Contextual Recall
10. Synthetic data generation can auto-create test datasets from your documents
11. CI/CD integration is native — add eval as a quality gate in GitHub Actions
12. Custom metrics are easy to build by subclassing BaseMetric
13. Confident AI provides optional cloud dashboards and team collaboration
14. Avoid evaluating with the same model you're testing — use a stronger judge LLM

Й Start Evaluating Today

```
pip install -U deepeval
```

github.com/confident-ai/deepeval | deepeval.com/docs